

astrid-runtime/astrid

# Astrid : un runtime capability-secure Rust/WASM techniquement solide, porté par une seule personne — adopter en périphérie, ne pas en dépendre encore

Astrid is a portable, capability-secure operating system for composable software.

1

BUS FACTOR  
sur 12 semaines

319

COMMITTS  
sur 12 semaines

117

DÉPENDANCES  
15 en production

Apache-2.0

LICENCE

<https://github.com/astrid-runtime/astrid> Licence Apache-2.0 Onboard · profil cto

## Que fait ce projet et quelle est son architecture ?

Un runtime capability-secure Rust/WASM de 33 crates, 10277 étoiles depuis sa création le 2026-02-15, encore « the hosted runtime, not a bootable OS image » : l'architecture est claire, le produit n'est pas fini <sup>1 2 3 4</sup>.

|                             |                                                          |
|-----------------------------|----------------------------------------------------------|
| <code>.cargo/</code>        | config Cargo locale                                      |
| <code>.gemini/</code>       | config agent Gemini                                      |
| <code>.github/</code>       | 20 workflows GitHub Actions                              |
| <code>changes/</code>       | notes de changement en attente                           |
| <code>container/</code>     | définitions de conteneur                                 |
| <code>crates/</code>        | code principal : 33 crates Rust, 32 membres du workspace |
| <code>docs/</code>          | documentation                                            |
| <code>e2e/</code>           | scénarios TOML et fixtures bout en bout                  |
| <code>fuzz/</code>          | tests de fuzzing                                         |
| <code>native/</code>        | code natif macOS                                         |
| <code>release/</code>       | config de release (nightly.toml)                         |
| <code>scripts/</code>       | scripts build, CI, release                               |
| <code>.dockerignore</code>  | exclusions du build Docker                               |
| <code>.gitattributes</code> | attributs git                                            |
| <code>.gitignore</code>     | exclusions git                                           |
| <code>.gitmodules</code>    | sous-modules git                                         |

... et 12 autres entrées, voir sources/tree.md

Astrid se présente comme « a portable, capability-secure operating system for composable software » <sup>5</sup> : aujourd'hui un runtime user-space macOS et Linux, la cible annoncée étant un OS autonome ; la release courante est « the hosted runtime, not a bootable OS image » <sup>4</sup>. Sa thèse : un agent LLM est du code non fiable, la frontière de sécurité doit être dans le runtime, pas dans le prompt <sup>4</sup>.

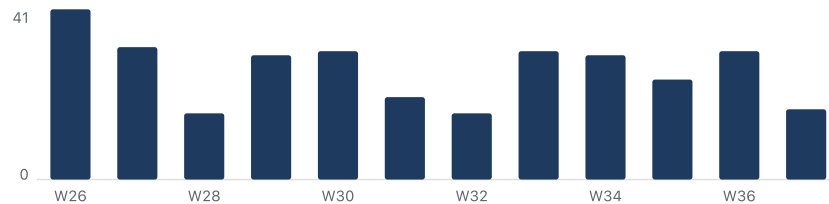
Architecture : un noyau volontairement « dumb » ( `astrid-daemon` ) qui charge des **capsules** WebAssembly dans Wasmtime, route l'IPC sous ACL de capacités et tient une chaîne d'audit signée ; les frontends (CLI, passerelle HTTP, Discord) sont des **uplinks** sur socket Unix ; l'ABI hôte est le component model avec des paquets WIT `astrid:*` ( `fs` , `io` , `kv` , `ipc` , `net` , `http` , `sys` , `process` , `approval` , `identity` , `elicit` , `uplink` ) <sup>4</sup>. Sécurité décomposée en gates indépendants et fail-closed : sandbox WASM, manifeste allow-list, ACL IPC, jeton ed25519, approbation, sandbox OS (bwrap/seatbelt), audit hash-linké <sup>4</sup>.

Code : un workspace Cargo de 32 membres, 33 crates sous `crates/` <sup>1 6</sup>, découpés par domaine : noyau ( `astrid-kernel` , `astrid-daemon` , `astrid-runtime` , `astrid-events` ), sécurité ( `astrid-crypto` , `astrid-capabilities` , `astrid-audit` , `astrid-approval` , `astrid-vfs` ), stockage ( `astrid-storage` + providers FUSE/FSKit/WinFsp), frontends ( `astrid-cli` , `astrid-gateway` , `astrid-mcp` , `astrid-uplink` ), outillage capsules ( `astrid-build` , `astrid-capsule*` , `astrid-emit` ) <sup>7</sup>. Autour : 20 workflows GitHub Actions <sup>8</sup>, `e2e/` en scénarios TOML <sup>9</sup>, `fuzz/` <sup>10</sup>, `native/macos/` <sup>11</sup>, un `CHANGELOG.md` de 410 Ko <sup>12</sup>.

Chiffres : 10277 étoiles <sup>2</sup>, 137 forks <sup>13</sup>, 216 issues ouvertes <sup>14</sup>, dépôt créé le 2026-02-15 <sup>3</sup>, organisation <sup>15</sup>, non archivé <sup>16</sup>. Le SDK des capsules ( `astrid-sdk` ), le Book, le handbook et les RFC sont dans d'autres dépôts <sup>17 18</sup> : le cache ne les couvre pas.

## Quelle est sa santé technique ?

16 à 41 commits par semaine, une CI qui bloque sur `clippy -D warnings` et `cargo-audit`, v2026.9.0 livrée le 2026-09-09 : le pipeline est sain, mais couverture et taux de succès des runs restent hors cache <sup>19 20 21 22</sup>.



Cadence soutenue et régulière : entre 16 commits <sup>19</sup> et 41 commits <sup>20</sup> par semaine sur les 12 dernières semaines, dernier commit le 2026-09-09 <sup>23</sup>. 149 issues fermées sur 30 jours <sup>24</sup> pour 192 ouvertes <sup>25</sup> ; 188 PR fusionnées en 30 jours listées <sup>26</sup>, 24 PR ouvertes <sup>27</sup>.

Releases : v2026.9.0 le 2026-09-09 <sup>22 28</sup>, après une série v0.9.3 → v0.10.4 en juillet <sup>29 30</sup> et trois canaux `channel-stable` / `nightly` / `dev` <sup>31 32 33</sup>. Le saut 0.10 → 2026.9 est un changement de schéma de version (`year.month.patch`) annoncé dans le README <sup>4</sup>.

CI : `ci.yml` déclenché sur push et pull\_request <sup>34 35</sup>, 14 jobs : `check`, `fmt`, `clippy -D warnings`, `cargo test --workspace --locked` sur ubuntu et macos, MSRV 1.95, `cargo-audit 0.22.2`, builds release Linux gnu/musl 4 cibles, tests natifs Windows (WinFsp épinglé par SHA256), portabilité wasm32, `cargo-semver-checks` et `cargo-public-api` consultatifs sur PR, `harden-runner` sur chaque job <sup>21</sup>. Vingt workflows au total dont `codeql`, `scorecard`, `dependency-review`, `nightly`, `release` <sup>21</sup>. Le README annonce `#![deny(unsafe_code)]` partout sauf `astrid-sys` et `astrid-sdk`, `clippy` pedantic, lint d'overflow arithmétique en erreur, binaires signés Sigstore keyless <sup>4</sup> ; le `Cargo.toml` confirme `unsafe_code = "deny"` et `arithmetic_side_effects = "deny"` <sup>6</sup>.

Tests : `crates/astrid-integration-tests`, 39 fichiers, `cargo test + insta` <sup>36 37 38</sup>, commande `cargo test --workspace -- --quiet` <sup>39</sup>. Le cache ne contient ni couverture, ni durée de CI, ni taux de succès des runs ; à vérifier sur <https://github.com/astrid-runtime/astrid/actions>.

Bémol : Windows est testé en CI mais exclu des archives 2026.9.0 <sup>4</sup> ; `wit` à la racine est une entrée de 0 octet <sup>40</sup> et `.gitmodules` déclare des sous-modules dont le cache ignore la cible <sup>41</sup>.

## Qui porte le projet et quel est le bus factor ?

Bus factor 1 : joshuajbouw signe 309 commits sur 12 semaines contre 7 pour le deuxième contributeur, risque high, aucun financement déclaré ; le fork miroir est une assurance, pas une option <sup>42 43 44 45 46</sup>.



Bus factor : 1 <sup>42</sup>. Sur 12 semaines, joshuajbouw signe 309 commits <sup>47 43</sup>, jvsteiner 7 <sup>44</sup>, Copilot 3 <sup>48</sup>. Il n'y a que trois contributeurs dans le top 10 <sup>49</sup>.

Le Cargo.toml nomme deux auteurs : « Joshua J. Bouw » et « Unicity Labs » <sup>6</sup> ; le dépôt appartient à une organisation <sup>15</sup> et un CODEOWNERS existe <sup>50</sup>. Le cache ne dit pas qui d'autre est mainteneur ni la nature du lien avec Unicity Labs ; à vérifier dans <https://github.com/astrid-runtime/astrid/blob/main/.github/CODEOWNERS> et <https://github.com/astrid-runtime/handbook>.

Le risque est classé haut par le collecteur : « Un seul contributeur porte la moitié des commits sur 12 semaines » <sup>45 51</sup>. Aucun financement déclaré (pas de FUNDING) <sup>46</sup>. Parade : ne pas dépendre d'un fork privé, mais sécuriser un accès aux clés de release et aux dépôts satellites, ou négocier un contrat de maintenance avec l'éditeur.

## Quelles dépendances et quelle dette ?

117 dépendances workspace avec Wasmtime 48 en pivot, 91 issues `needs-design` et 0 repère IA sur 7 : la dette est dans les contrats non gelés, pas dans les TODO <sup>52</sup>  
<sup>6 53 54 55</sup>.

Manifeste `Cargo.toml` <sup>56</sup>, 117 entrées dans `[workspace.dependencies]` <sup>52</sup> : 27 crates internes et 90 externes, dont 8 épinglées en version exacte ; `Cargo.lock` versionné (217 Ko) ; transitives non relevées <sup>57 58</sup>. Niveau de risque déclaré : mid <sup>59</sup>.

Dépendances structurantes : `wasmtime` / `wasmtime-wasi` 48.0.1, la famille `wasm-tools` 0.257.1 ( `wasmparser` , `wasm-encoder` , `wit-component` , `wit-parser` , `wat` épinglé `=1.257.1` ), `tokio` 1.49, `axum` 0.8, `rustls` 0.23, `ed25519-dalek` 3.0, `blake3` 1.5, `surrealkv` 0.21, `rmcp` ~3.1.4, `sigstore-verify` `=0.11.0` , `landlock` 0.4, et des crates de chunking peu diffusées épinglées ( `fastcdc` `=5.0.0` et `=4.0.1` en parallèle, `mincdc` `=0.1.0` , `mothcdc` `=0.7.2` ) <sup>6</sup>. Exemple d'épingle en cache : `caseless` `=0.2.2` <sup>60</sup>.

Lecture CTO : le couplage à Wasmtime est le plus lourd (montée 47 → 48 faite en août, PR #1786 et #1739 <sup>61 62</sup>) ; deux versions de `fastcdc` cohabitent et trois crates CDC exotiques verrouillées signalent un format de stockage en cours de stabilisation. Deux PR Dependabot attendent : 7 mises à jour d'actions <sup>63</sup> et 14 de crates Rust <sup>64</sup>.

Dette explicite : 0 résultat pour `TODO OR FIXME` mais 13 fichiers pour `TODO` seul, dont `crates/astrid-kernel/src/lib.rs` , `crates/astrid-vfs/src/workspace_cow/overlayfs.rs` et le WIT `astrid-net@1.0.0` <sup>65</sup> ; `FIXME` non compté <sup>65</sup>. Dette implicite : 91 issues `needs-design` <sup>53</sup>, 16 `blocked` <sup>66</sup>, 42 `bug` <sup>67</sup> ; aucun repère IA (AGENTS.md, CLAUDE.md...) : 0 sur 7 <sup>54</sup>  
<sup>55</sup>.

## Quels risques ?

Un seul risque high, le bus factor 1 ; dépendances mid, licence low, CI low ont chacun une parade, et le `SECURITY.md` figé à 0.2.x face à v2026.9.0 est le signal à surveiller <sup>45 42 59 68 69 70 22</sup>.

- **license** Apache-2.0 : licence permissive.

---

- **bus\_factor** Un seul contributeur porte la moitié des commits sur 12 semaines.

---

- **deps** 117 entrées dans `[workspace.dependencies]` de `Cargo.toml` : 27 crates internes (path) et 90 crates externes, dont 8 épinglées en version exacte (=). `Cargo.lock` versionné (217 Ko). Nombre de dépendances transitives non relevé.

---

- **ci** 20 workflows dans `.github/workflows.ci.yml` (push sur main et `stream-*`, `pull_request`) enchaîne `cargo check`, `fmt`, `clippy -D warnings`, `cargo test --workspace` (ubuntu et macos), `MSRV 1.95`, `cargo-audit`, builds release Linux gnu/musl, tests natifs Windows, portabilité `wasm32`.

---

Quatre risques relevés par le collecteur <sup>71</sup> : licence low <sup>68</sup>, bus factor high <sup>45</sup>, dépendances mid <sup>59</sup>, CI low <sup>69 72</sup>.

**Bus factor — HIGH.** Un seul auteur pour 309 des commits <sup>43</sup>, bus factor 1 <sup>42</sup>. Parade : conserver un fork miroir et une build reproductible interne, suivre l'organisation plutôt que la personne.

**Sécurité déclarée vs prouvée — MID.** Périmètre bien cadré dans `SECURITY.md` (échappement de sandbox, forge de jetons, `ed25519/blake3`, SSRF, audit), réponse sous 48 h et correctif sous 7 jours <sup>70 73</sup>. Mais le tableau « Supported Versions » s'arrête à 0.2.x <sup>70</sup> alors que la release est 2026.9.0 <sup>22</sup> : politique non tenue à jour. 46 issues portent le label `security` <sup>74</sup> et l'architecture « minimal kernel vs privileged package services » est encore en discussion (#1719, p1, needs-design) <sup>75 76</sup>. Le cache ne contient aucun audit externe ; à vérifier dans <https://github.com/astrid-runtime/astrid/security>.

**Stabilité des contrats — MID.** Trois campagnes en cours ( `campaign/os-universal` 32 issues <sup>77</sup>, `campaign/station` ), 91 issues `needs-design` <sup>53</sup>, contrats (ABI, IPC, manifeste, SDK) soumis à RFC <sup>4</sup>, semver-checks seulement consultatif <sup>21</sup>. Parade : épingler une release, ne consommer que l'ABI WIT documentée.

**Plateformes — MID.** Windows hors archives 2026.9.0 ; macOS exige FSKit sur macOS 26+ et une approbation d'extension signée <sup>4</sup>. Parade : cibler Linux GNU/MUSL en premier.

**Licence — LOW mais à clarifier.** GitHub relève Apache-2.0 <sup>78</sup> ; le README et le `Cargo.toml` disent « MIT OR Apache-2.0 » <sup>4 6</sup>, les deux fichiers sont présents <sup>79 80</sup>. Double licence permissive, pas de blocage.

**Dépendances — MID.** Voir section précédente ; `cargo-audit` et `dependency-review` en CI <sup>21</sup> limitent l'exposition.

## Adopter, contribuer ou forker ?

Adopter en pilote non critique sur Linux, version épinglée v2026.9.0, fork miroir en assurance ; réévaluer quand le bus factor dépasse 1 et que les contrats sortent des RFC <sup>22 42 4</sup>.

Roadmap : pas de jalons dans le cache ( `milestones` absent) et thèmes vides <sup>81</sup> ; à vérifier dans <https://github.com/astrid-runtime/astrid/milestones>. Ce qui se lit : 10 PR ouvertes listées dont 7 en draft <sup>82 83</sup>, et les demandes les plus discutées vont vers l'OS natif et la station de capsules : #1708 « qualify the signed amd64 Linux-backed Astrid host profile » (29 commentaires) <sup>84 85</sup>, #1564 « epic: build the Astrid universal application substrate » <sup>86</sup>, #1406 namespaces de capacités dynamiques (blocked) <sup>87 88</sup>, #1664 cérémonie d'enrôlement du premier propriétaire (blocked) <sup>89</sup>. Le projet investit dans son futur OS, pas dans la consolidation du runtime hébergé.

**Adopter** : oui pour un usage en périphérie (sandbox d'outils d'agents, passerelle MCP), sur Linux, version épinglée, avec build interne reproductible ; pas comme socle d'un produit tant que le bus factor reste à 1 <sup>42</sup> et que les contrats sont sous RFC <sup>4</sup>.

**Contribuer** : possible mais encadré, workflow issue-first, tiers de contributeurs, crates de sécurité réservées au core, refactors externes refusés <sup>4 90</sup> ; le seul relecteur effectif est aussi l'auteur principal <sup>47</sup>. Utile pour faire entrer des correctifs de plateforme, pas pour peser sur l'architecture.

**Forker** : la double licence <sup>6</sup> le permet, mais reprendre 33 crates <sup>1</sup>, 117 dépendances <sup>52</sup> et une cadence de 16 à 41 commits/semaine <sup>19 20</sup> sans l'auteur n'est pas réaliste à court terme.

**Recommandation** : adopter en pilote non critique dès maintenant, contribuer sur les tests et la plateforme Linux pour gagner le tier Astronaut <sup>90</sup>, garder un fork miroir comme assurance ; réévaluer dans deux releases sur trois signaux : bus factor > 1, `SECURITY.md` à jour, et RFC des contrats gelés.

## Décision

- **Adopter en périphérie, ne pas en dépendre.** Astrid entre comme sandbox d'outils d'agents ou passerelle MCP sur Linux GNU/MUSL, jamais comme socle d'un produit tant que le bus factor reste à 1 <sup>42</sup> <sup>4</sup>.
- **Conditions d'entrée** : release épinglée v2026.9.0 <sup>22</sup>, build interne reproductible à partir du `Cargo.lock` versionné <sup>57</sup>, binaires vérifiés via Sigstore keyless <sup>4</sup>, fork miroir tenu à jour sous double licence MIT OR Apache-2.0 <sup>6</sup>.
- **Contribution ciblée** : tests et plateforme Linux, en issue-first, pour atteindre le tier Astronaut ; aucun effort sur les crates de sécurité, réservées aux tiers Core et Maintainer <sup>90</sup>.
- **Réévaluer dans deux releases sur quatre signaux** : bus factor > 1 <sup>42</sup>, `SECURITY.md` aligné sur la version courante (0.2.x aujourd'hui, contre v2026.9.0) <sup>70</sup> <sup>22</sup>, contrats ABI/IPC/SDK sortis du processus RFC <sup>4</sup>, reflux des 91 issues `needs-design` <sup>53</sup>.
- **Ne pas forker** : reprendre 33 crates, 117 dépendances et 16 à 41 commits par semaine sans l'auteur principal n'est pas tenable <sup>1</sup> <sup>52</sup> <sup>19</sup> <sup>20</sup>.



## NOTES

### Sources

1. donnée collectée `tree.18.role`
2. donnée collectée `repo.stars`
3. donnée collectée `repo.created_at`
4. source du repo `readme.md`
5. donnée collectée `repo.description`
6. source du repo `manifest.md`
7. source du repo `tree.md`
8. donnée collectée `tree.4.role`
9. donnée collectée `tree.20.role`
10. donnée collectée `tree.21.role`
11. donnée collectée `tree.22.role`
12. donnée collectée `tree.7.role`
13. donnée collectée `repo.forks`
14. donnée collectée `repo.open_issues`
15. donnée collectée `repo.owner_type`
16. donnée collectée `repo.archived`
17. GitHub <https://github.com/astrid-runtime/sdk-rust>
18. GitHub <https://github.com/astrid-runtime/rfcs>
19. donnée collectée `activity.commits_per_week.2.count`
20. donnée collectée `activity.commits_per_week.0.count`
21. source du repo `ci.md`
22. donnée collectée `releases.0.tag`
23. donnée collectée `activity.last_commit`
24. donnée collectée `issues.closed_30d`
25. donnée collectée `issues.open`
26. donnée collectée `pulls.merged_30d`
27. donnée collectée `pulls.open`
28. donnée collectée `releases.0.date`
29. donnée collectée `releases.9.tag`
30. donnée collectée `releases.1.tag`
31. donnée collectée `releases.5.tag`
32. donnée collectée `releases.6.tag`
33. donnée collectée `releases.7.tag`
34. donnée collectée `build.ci.0.path`
35. donnée collectée `build.ci.0.triggers.0`
36. donnée collectée `tests.dir`
37. donnée collectée `tests.files`
38. donnée collectée `tests.framework`
39. donnée collectée `build.test`
40. donnée collectée `tree.27.role`
41. donnée collectée `tree.6.role`
42. donnée collectée `activity.bus_factor`
43. donnée collectée `activity.contributors.0.commits`
44. donnée collectée `activity.contributors.1.commits`
45. donnée collectée `risks.1.level`
46. donnée collectée `business.funding`
47. donnée collectée `activity.contributors.0.login`
48. donnée collectée `activity.contributors.2.commits`
49. donnée collectée `activity.contributors`
50. donnée collectée `business.codeowners`
51. donnée collectée `risks.1.note`
52. donnée collectée `deps.count`
53. donnée collectée `issues.by_label.needs-design`
54. donnée collectée `ai_docs.score`
55. donnée collectée `ai_docs.max`
56. donnée collectée `deps.manifest`
57. donnée collectée `risks.2.note`
58. donnée collectée `tree.9.role`

59. donnée collectée risks.2.level

60. donnée collectée deps.runtime.11.version

61. donnée collectée pulls.merged\_30d.52.number

62. donnée collectée pulls.merged\_30d.82.number

63. donnée collectée roadmap.open\_prs.0.title

64. donnée collectée roadmap.open\_prs.1.title

65. source du repo todo.md

66. donnée collectée issues.by\_label.blocked

67. donnée collectée issues.by\_label.bug

68. donnée collectée risks.0.level

69. donnée collectée risks.3.level

70. source du repo security.md

71. donnée collectée risks

72. donnée collectée risks.3.note

73. donnée collectée business.security\_policy

74. donnée collectée issues.by\_label.security

75. donnée collectée roadmap.requests.4.title

76. donnée collectée roadmap.requests.4.labels.2

77. donnée collectée issues.by\_label.campaign/os-universal

78. donnée collectée repo.license

79. donnée collectée tree.11.path

80. donnée collectée tree.12.path

81. donnée collectée roadmap.themes

82. donnée collectée roadmap.open\_prs

83. donnée collectée roadmap.open\_prs.2.draft

84. donnée collectée roadmap.requests.0.title

85. donnée collectée roadmap.requests.0.comments

86. donnée collectée roadmap.requests.7.title

87. donnée collectée roadmap.requests.2.title

88. donnée collectée roadmap.requests.2.labels.2

89. donnée collectée roadmap.requests.6.title

90. source du repo contributing.md